

CIA

VM1 Security Audit Report

Web Frontend

May 2026

Version 1.0

Team Members:

Muhammad Noor Afaqi

Subhan Khalik

Shanzay Asad

Table of Contents

Overview.....	3
1. Reconnaissance	3
1.1 Port Scan	3
1.2 Hidden directory discovered	3
1.3 nginx 403 + OS Fingerprinting.....	4
1.4 React App (Port 8080).....	4
2. Directory Enumeration	4
2.1 gobuster on /WorkInProgress/	4
3. Vulnerabilities.....	5
1. Root SSH Access via provided password	5
2. Exposed Hidden PHP App via robots.txt	5
3. Publicly Readable Credential Store (userlist.txt + README.txt)	6
4. Outdated nginx 1.16.1 + Version Disclosure	7
5. Kernel 3.10.0-327 - Dirty COW (CVE-2016-5195)	7
6. FTP Service (proftpd) Exposed on Port 21	8
7. API IP Hardcoded in React JS Bundle.....	8
4. Tools Used.....	9

Overview

Field	Detail
Local IP	192.168.64.5
OS	CentOS/RHEL 7 and Fedora base (kernel 3.10.0-327.el7.x86_64)
Services found	SSH:22, nginx:80, React App:8080, FTP(proftpd):21
Containers	front_front_1, m1_nginx_1, m1_flatfile_1, m1_proftpd_1
Date	27/05/2026
Tester	Techies
Result	Full access recovered

Note: Local IPs for each VM can be different when run on different machines

1. Reconnaissance

1.1 Port Scan

Command used:

```
nmap -sV -sC -p- --open 192.168.64.5
```

Port	Finding
22/tcp	OpenSSH (open)
80/tcp	nginx 1.16.1 — returns 403 on root
8080/tcp	React SPA (create-react-app, served via custom binary)
21/tcp	FTP, proftpd (discovered post-access via docker ps)
443,3000,3306,5432,9000	All closed on this VM

Root access was gained on the system.

1.2 Hidden directory discovered

```
curl -s http://192.168.64.5:80/robots.txt
=> Disallow: /WorkInProgress/%
```

The nginx server on port 80 exposes a robots.txt that explicitly disallows /WorkInProgress/, revealing a hidden PHP application not linked from the main site.

1.3 nginx 403 + OS Fingerprinting

```
curl -v http://192.168.64.5:80
```

```
curl -s http://192.168.64.5:80/.env
```

Root returns 403 Forbidden. The default Fedora 404 page (returned for unknown paths) leaks OS identity via `/usr/share/nginx/html/404.html` path and Fedora logo reference.

1.4 React App (Port 8080)

```
curl -s http://192.168.64.5:8080/manifest.json
```

Standard create-react-app build.

No sensitive data in `manifest.json`, `.env`, or `.git/config` — all return the SPA catch-all `index.html`.

2. Directory Enumeration

2.1 gobuster on /WorkInProgress/

```
gobuster dir -u http://192.168.64.5:80/WorkInProgress/ -w ~/common.txt -x txt,html,php,json,conf,log,bak,zip -t 50
```

File	Notes
<code>index.php / control.php</code>	User Control Panel and login portal
<code>login.php</code>	POST login handler
<code>join.php</code>	Registration <code>freeJoin=0</code> , disabled
<code>logout.php</code>	Session logout
<code>adduser.php</code>	Admin user creation (auth required)
<code>deluser.php</code>	Admin user deletion (auth required)
<code>edituser.php / changepass.php</code>	Password management (auth required)
<code>config.php</code>	Returns empty — server-side include only
<code>README.txt</code>	PUBLIC therefore reveals app name, author, default creds, file paths
<code>userlist.txt</code>	PUBLIC therefore live credential database with MD5 hashed passwords

3. Vulnerabilities

1. Root SSH Access via provided password

Attribute	Detail
Severity	CRITICAL
Location	SSH — port 22 — root account
Credential	root : <no password>
Status	EXPLOITED

The root SSH password was present in plaintext in the leaked .creds.txt file.

Direct root login was possible with no additional exploits required.

File was not used, you can simply use enter to log in.

Commands used:

```
nmap -p 22 192.168.64.5
```

```
ssh root@192.168.64.5
```

```
whoami => root
```

```
uname -r => 3.10.0-327.el7.x86_64
```

Applied Fixes:

- Disable root SSH login: set 'PermitRootLogin no' in /etc/ssh/sshd_config
- Rotate all credentials, treat every password in .creds.txt as compromised
- Enforce SSH key-based authentication only, disable password auth
- Remove .creds.txt from the repository and purge from git history

2. Exposed Hidden PHP App via robots.txt

Attribute	Detail
Severity	MEDIUM
Location	http://192.168.64.5:80/WorkInProgress/
Status	Gained access to admin panel

The robots.txt Disallow directive for /WorkInProgress/ directly advertised the path to a hidden PHP user management application. Directory enumeration then revealed the full file structure.

```
curl -s http://192.168.64.5:80/robots.txt => Disallow: /WorkInProgress/%
gobuster dir -u http://192.168.64.5:80/WorkInProgress/ ...
```

Patch:

- Remove the Disallow entry from robots.txt, it reveals more items
- Restrict /WorkInProgress/ via nginx allow/deny or move it outside the web root.

3. Publicly Readable Credential Store (userlist.txt + README.txt)

Attribute	Detail
Severity	HIGH
Location	http://192.168.64.5:80/WorkInProgress/userlist.txt
Location 2	http://192.168.64.5:80/WorkInProgress/README.txt
Status	Credential file overwritten

Both files were publicly readable with no authentication. README.txt disclosed the application source, default credentials (admin/admin, test/test), and the credential file path. userlist.txt contained the live MD5-hashed password database.

```
curl -s http://192.168.64.5:80/WorkInProgress/userlist.txt
=> 2f7f260f30d13a6f948206c297de79601admin
format: MD5_hash(32) + role_flag(1) + username
```

Steps used when SSH was gained.

```
# Generate MD5 hash for new password
python3 -c "import hashlib; print(hashlib.md5(b'pwned').hexdigest())"

# Overwrite credential file inside container
echo '5e93de3efa544e85dcd6311732d28f951admin' > /tmp/data/userlist.txt

# Login to admin panel
curl -X POST http://192.168.64.5:80/WorkInProgress/login.php -d 'username=admin&password=pwned'
=> You have been logged in.
```

Patch:

- Move userlist.txt outside the web root (the README itself recommends this).
- Remove README.txt from the web-accessible directory.
- Replace MD5 with bcrypt (cost ≥ 12) or Argon2id.
- Replace this 2003 flat-file system with a database-backed auth solution.

4. Outdated nginx 1.16.1 + Version Disclosure

Attribute	Detail
Severity	HIGH
Location	:80
Version	nginx/1.16.1 (EOL — multiple known CVEs)
Status	Out of date

```
curl -I http://192.168.64.5:80 => Server: nginx/1.16.1
```

nginx 1.16.1 is end-of-life. The exact version is exposed in every response header and in default error pages, enabling targeted CVE exploitation. Additionally, all packages are 6 years old and entirely out of date

Patch:

- Update nginx to 1.24.x where x is a stable version of choice or latest stable inside the Docker container.
- Add 'server_tokens off;' to nginx.conf to suppress version in headers and error pages.
- Replace the default Fedora 404/50x error pages with custom ones.

5. Kernel 3.10.0-327 - Dirty COW (CVE-2016-5195)

Attribute	Detail
Severity	CRITICAL
Location	VM1 host kernel
Kernel	3.10.0-327.el7.x86_64 — vulnerable (fix requires >= 4.8.3)
Status	Kernel can be exploited by COW

```
uname -r => 3.10.0-327.el7.x86_64
```

The running kernel is vulnerable to Dirty COW (CVE-2016-5195), a race condition in the copy-on-write mechanism that allows any local user to gain root via write access to read-only memory mappings. In this case root SSH was already obtained via leaked credentials, but we used Dirty COW to create an additional root user.

Exploit steps (from low-priv shell):

```
gcc -pthread dirty.c -o dirty -lcrypt
```

```
./dirty [newpassword]
su fireart # or su with new root password
```

Patch:

- Update kernel to $\geq 4.8.3$ via `yum update kernel` inside the VM.
- If immediate kernel update is not possible, apply grsecurity/PaX hardening patches.
- Restrict local user accounts to minimum required privileges (principle of least privilege).

6. FTP Service (proftpd) Exposed on Port 21

Attribute	Detail
Severity	MEDIUM
Location	Port 21 — m1_proftpd_1 container
Status	Credentials bruteforced

An FTP service (proftpd) is running and exposed on port 21. This was not visible in the initial nmap scan but discovered via `docker ps` after SSH access. FTP transmits credentials in plaintext and is a known attack possibility.

```
docker ps => m1_proftpd_1 proftpd -n 0.0.0.0:21->21/tcp
```

Patch:

- Remove proftpd entirely if FTP is not required for the application.
- If FTP is needed, replace it with SFTP (SSH file transfer) or FTPS (FTP over TLS).
- Ensure FTP does not accept any of the leaked credentials from `.creds.txt`.

7. API IP Hardcoded in React JS Bundle

Attribute	Detail
Severity	MEDIUM
Location	React bundle — main.5a348464.chunk.js
Exposed data	<code>http://10.128.173.127:3000</code> — VM2 API IP and port
Status	Identified — exposes internal network topology

The compiled React JS bundle contains a hardcoded internal IP for the API backend. Anyone with access to the bundle can extract the internal address of VM2 and probe it directly.

```
grep -oE 'https?://[0-9]+\.[0-9]+\.[0-9]+[^\"]*' main.5a348464.chunk.js
=> http://10.128.173.127:3000 (found twice)
```


Patch:

- Use relative API paths (/api/...) proxied via nginx — never hardcode internal IPs in client-side code.
- Rebuild the React app with REACT_APP_API_URL pointing to a domain or relative URL.

4. Tools Used

Tool	Command	Purpose
nmap	nmap -sV -sC -p- --open	Port scan, service/version detection
curl	curl -s -v -I -X POST -c/-b	HTTP probing, header inspection, login, cookie handling
gobuster	gobuster dir -w common.txt -x php,txt	Directory/file brute force on /WorkInProgress/
Python3 hashlib	hashlib.md5(p.encode()).hexdigest()	MD5 hash generation for credential file manipulation
Python3 urllib	urllib.request POST loop	Credential brute force (hydra replacement)
ssh	ssh root@192.168.64.5	Remote shell access using leaked credentials
docker	docker ps / docker inspect / docker exec	Container enumeration and file system access
SecLists	common.txt, 10k/100k wordlists	Directory enum and hash cracking wordlists
crackstation.net	Online MD5 rainbow table	Hash cracking attempt (not found)